

Code Explanation

DLT Pipeline Code Explanation

The provided code defines a DLT (Delta Live Tables) pipeline for processing customer and order data. The pipeline consists of four tables: customer, order, ordersummary, and customeraggregatespend. The code is written in Python and utilizes the PySpark library for data processing.

Overview of the Code

The DLT pipeline is designed to process customer and order data from CSV files, apply data quality rules, and perform aggregations. The pipeline consists of four main steps: reading and processing customer and order data, creating an ordersummary table with SCD Type 2 logic, and calculating aggregated customer spending.

Step 1: Reading and Processing Customer Data

The first step involves reading customer data from a CSV file and applying data quality rules. The data is read using the `spark.read.csv()` method, and duplicates are dropped using the `dropDuplicates()` method.

```
return (spark.read
        .option("header", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")
        .dropDuplicates()
        .filter(col("CustId").isNotNull() &
                col("Name").isNotNull() &
                col("EmailId").isNotNull() &
                col("Region").isNotNull()))
```

Step 2: Reading and Processing Order Data

The second step involves reading order data from a CSV file, applying data quality rules, and calculating the TotalAmount.

```
return (spark.read
        .option("header", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a")
        .dropDuplicates()
        .filter(col("OrderId").isNotNull() &
                col("ItemName").isNotNull() &
                col("PricePerUnit").isNotNull() &
                col("Qty").isNotNull() &
                col("Date").isNotNull() &
                col("CustId").isNotNull())
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
        )
```

Step 3: Creating the Ordersummary Table with SCD Type 2 Logic

The third step involves creating the ordersummary table by joining the customer and order data. The table is designed to maintain a history of changes to customer attributes using SCD Type 2 logic.

```
# Join the data
joined_df = (customer_df.join(order_df, "CustId")
               .select("CustId", "Name", "EmailId", "Region",
                       "OrderId", "ItemName", "PricePerUnit", "Qty", "
Date"))
```

For the initial load, the code creates the ordersummary table with SCD2 fields.

```
return (joined_df
        .withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp())
        .withColumn("EndDate", lit(None).cast(TimestampType())))
```

For subsequent loads, the code implements SCD Type 2 logic by identifying records with changes in customer attributes, marking existing records as inactive, and creating new active records.

```
# Identify records with changes in customer attributes
changed_customers = (
    joined_df.alias("source")
    .join(
        existing_df.alias("target").filter(col("IsActive") == True),
        "CustId"
    )
    .where(
        (col("source.Name") != col("target.Name")) |
        (col("source.EmailId") != col("target.EmailId")) |
        (col("source.Region") != col("target.Region"))
    )
    .select("source.CustId")
    .distinct()
)

# Mark existing records for these customers as inactive
existing_to_update = (
    existing_df
    .join(changed_customers, "CustId")
    .filter(col("IsActive") == True)
    .withColumn("IsActive", lit(False))
    .withColumn("EndDate", current_timestamp())
)

# Create new active records for these customers
new_records = (
    joined_df
    .join(changed_customers, "CustId")
    .withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp())
    .withColumn("EndDate", lit(None).cast(TimestampType()))
)
```

Step 4: Calculating Aggregated Customer Spending

The final step involves calculating aggregated customer spending by name and date.

```
# Perform aggregation
return (active_records
        .groupBy("Name", "Date")
        .agg({"TotalAmount": "sum"})
        .withColumnRenamed("sum(TotalAmount)", "TotalAmount")
        .select("Name", "TotalAmount", "Date"))
```